P.Vishwa Nandan(192473015)

TASK A4 – WHAT SHOULD BE SERIALIZED? (4 MARKS)

1. Aim

To understand Java object serialization and identify which fields should be stored when saving a Hostel object. The task also explains the use of the transient keyword, object-graph serialization, and serialVersionUID for maintaining compatibility when class definitions change.

2. Problem Description

A hostel management application stores its information using Java object serialization.

The application contains the following object structure:

- Hostel contains a list of Room objects.
- Each Room contains a list of Resident objects.
- Each Resident contains a name, register number, password, and database Connection reference.

The application must save the hostel information into a file and restore it later.

However, not every field should be serialized. Sensitive information and live database resources must be handled separately.

3. Task A4.1 – Decide What to Persist (2 Marks)

3.1 Fields That Should Be Serialized

Serialization converts an object's state into a byte stream so that it can be saved in a file or transferred.

The following fields should be serialized:

Class	Field	Serialize?	Reason
Hostel	Hostel name	Yes	Required to identify the hostel
Hostel	List of rooms	Yes	Required to restore hostel structure
Room	Room number	Yes	Identifies each room
Room	List of residents	Yes	Stores resident information
Resident	Name	Yes	Required resident information
Resident	Register number	Yes	Identifies the student

Resident	Phone number	Yes	Required contact information
Resident	Password	No	Sensitive authentication information
Resident	Database Connection	No	Live database resource that cannot be meaningfully restored through serialization

3.2 Use of the transient Keyword

Java provides the transient keyword to prevent a particular instance field from being serialized by the default serialization mechanism.

Example:

```
class Resident implements Serializable {

    String name;

    String registerNumber;

    transient String password;

    transient Connection connection;
}
```

In this example:

- name is serialized.
- registerNumber is serialized.
- password is not serialized.
- connection is not serialized.

Why Should Password Be transient?

A password is sensitive authentication information.

Storing a plaintext password in a serialized file can expose it if the file is accessed by an unauthorised person.

Therefore, the password should not be stored in the hostel object file.

Authentication information should be handled separately using a secure authentication system. Password verification should use an appropriately salted password hash rather than storing plaintext passwords.

Why Should Connection Be transient?

A database Connection represents a live connection to a database server.

It contains runtime resources such as network communication state and database-session information.

A Connection object cannot be meaningfully restored simply by reading it from a serialized file.

It must be excluded from serialization and recreated when required.

For example:

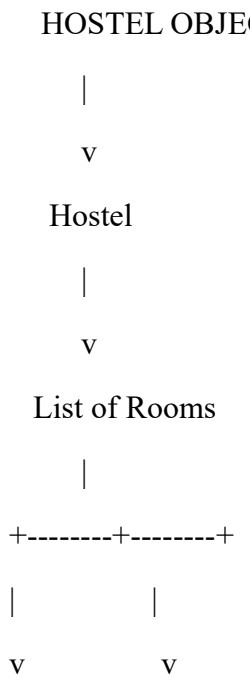
```
Connection connection =
   DriverManager.getConnection(
      URL, USER, PASSWORD
   );
```

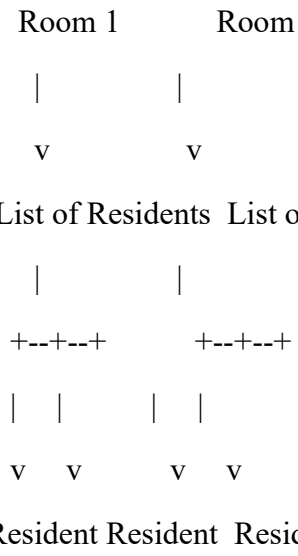
The application creates a new connection after restoring the hostel data.

3.3 Object-Graph Serialization

When the Hostel object is serialized, Java follows its references and serializes the reachable serializable objects.

For example:





The object graph contains all the objects reachable through the serializable references.

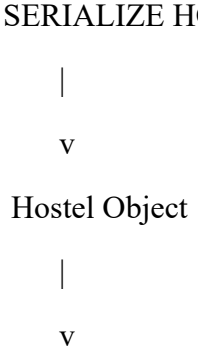
What Happens During Serialization?

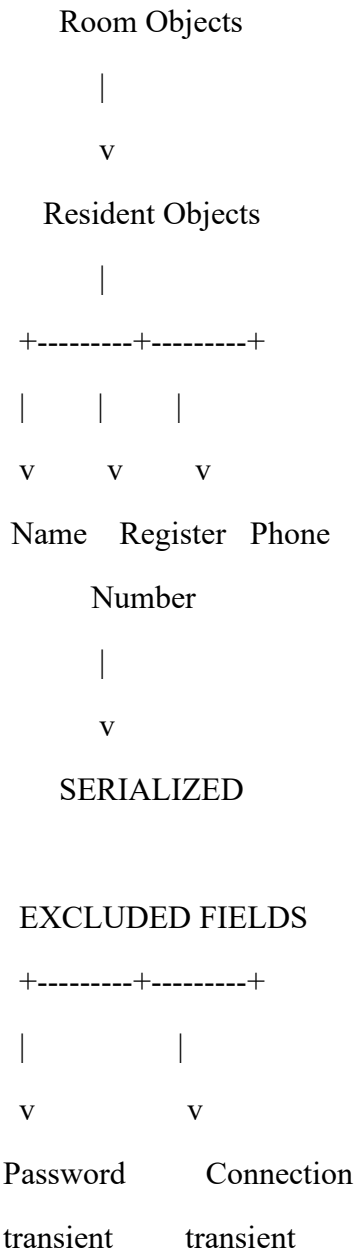
1. Java starts with the Hostel object.
2. It serializes the Hostel fields that are eligible for serialization.
3. It follows the reference to the list of Room objects.
4. It serializes each Room object.
5. It follows each Room's Resident list.
6. It serializes the Resident objects.
7. It skips fields marked transient.
8. The resulting object graph is written into a byte stream.

All classes in the serializable object graph must implement Serializable, unless a referenced field is excluded using transient or handled through custom serialization.

Java serialization also tracks object references, so shared objects and circular references can be handled without endlessly serializing the same object.

3.4 Object-Graph Serialization Diagram





4. Java Implementation – Hostel Serialization

4.1 Java Program – HostelSerializationDemo.java

4.2 Program Explanation

Step 1 – Implement Serializable

The Hostel, Room, and Resident classes implement the Serializable interface.

This makes their eligible object data available for serialization.

Step 2 – Mark Sensitive Fields

The password and database Connection fields are marked transient.

Therefore, they are excluded from default serialization.

Step 3 – Create the Object Graph

The program creates:

- One Hostel object.
- One Room object.
- One Resident object.

The Room object is added to the Hostel, and the Resident object is added to the Room.

Step 4 – Serialize the Object

ObjectOutputStream writes the Hostel object graph into hostel.dat.

Step 5 – Deserialize the Object

ObjectInputStream reads the stored data and reconstructs the Hostel, Room, and Resident objects.

Step 6 – Display Restored Information

The saved resident information is displayed.

The password and Connection are null because they were marked transient and were not stored.

4.3 Expected Output

Hostel data saved successfully.

Restored Hostel Information:

Hostel Name: SIMATS Hostel

Room Number: 101

Resident Name: Arun Kumar

Register Number: 192311001

Phone Number: 9876543210

Password after reading: null

Connection after reading: null

4.4 Important Observation

When Java deserializes an object, fields marked transient are assigned their default values unless the program explicitly restores them.

For the given fields:

- String password becomes null.
- Connection becomes null.

The application must separately authenticate the resident and establish a new database connection when required.

5. Task A4.2 – Plan for Change (2 Marks)

5.1 Adding a New Field to Resident

Suppose the original Resident class contains:

```
class Resident implements Serializable {

    String name;

    String registerNumber;

    transient String password;

    transient Connection connection;
}
```

After some time, the application introduces a new field:

```
String phoneNumber;
```

The updated class becomes:

```
class Resident implements Serializable {

    private static final long serialVersionUID = 1L;

    String name;

    String registerNumber;

    String phoneNumber;
```

```
    transient String password;


    transient Connection connection;
}
```

5.2 What Happens When an Old File Is Read?

Suppose a hostel file was created before the phone number field was introduced.

The old serialized file contains the previous Resident object structure.

When the new program reads that file, Java checks the class compatibility using serialVersionUID.

If the old and new versions have the same compatible serialVersionUID, Java can deserialize the object.

Since the old file does not contain a phoneNumber value, Java assigns the default value to the new field.

For a String field:

phoneNumber = null

Therefore, the old hostel data can still be read even though the new field was not present when the file was created.

5.3 Role of serialVersionUID

serialVersionUID is a unique version identifier used during Java serialization and deserialization.

Example:

private static final long serialVersionUID = 1L;

It helps Java determine whether a serialized object is compatible with the current class definition.

Case 1 – Same serialVersionUID

If the old and new classes use the same serialVersionUID and the change is compatible, Java can deserialize the old object.

The newly added phoneNumber field receives its default value.

Case 2 – Different serialVersionUID

If the serialVersionUID in the new class differs from the value stored in the serialized file, Java normally rejects deserialization.

It produces an exception similar to:

java.io.InvalidClassException:

Resident; local class incompatible

5.4 Comparison

Condition	Result
Same serialVersionUID, new String field added	Old file can be read; new field becomes null
Same serialVersionUID, new primitive field added	Old file can be read; field gets its default value
Different serialVersionUID	InvalidClassException may occur
Field marked transient	Field is not restored from the serialized data
Field removed	Old stored field is generally ignored if the class remains compatible

5.5 Providing a Default Value for the New Field

The application can provide a default value when an old file does not contain the phone number.

For example:

```
private void readObject(ObjectInputStream in)
      throws IOException, ClassNotFoundException {

   in.defaultReadObject();

   if (phoneNumber == null) {

      phoneNumber = "Not provided";
   }
}
```

The readObject() method performs default deserialization first and then checks the newly added field.

If it is missing, the program assigns a default value.

This makes the application more compatible with previously saved hostel files.

5.6 Importance of Version Management

1. Maintain an explicit serialVersionUID.
2. Keep it unchanged for compatible class changes.
3. Test old serialized files after modifying a class.
4. Provide default values for newly added fields where necessary.
5. Use a migration strategy for incompatible changes.
6. Never assume that the same serialVersionUID guarantees that every possible class change is safe.

6. Alternative Concept and Complexity Analysis

6.1 Alternative – CSV or JSON Text Storage

Instead of Java object serialization, the hostel management application can save resident information using CSV or JSON files.

Both formats store information as readable text.

Approach 1 – CSV Format

CSV stores records in rows, with fields separated by commas.

Example:

Hostel,Room,Name,Register Number,Phone Number

SIMATS Hostel,101,Arun Kumar,192311001,9876543210

SIMATS Hostel,101,Divya,192311002,9876543211

SIMATS Hostel,102,Ravi Kumar,192311003,9876543212

Sensitive passwords and live database connections are not included.

Advantages:

- Easy to read and edit.
- Supported by spreadsheet applications.
- Suitable for exporting resident information.
- Does not depend on Java-specific class definitions.

Disadvantages:

- Requires parsing and field validation.
- Special characters such as commas must be escaped.

- Nested object relationships are not naturally represented.
- Changes in column order can cause compatibility issues unless headers are used carefully.

Approach 2 – JSON Format

JSON represents information using objects, arrays, and key-value pairs.

Example:


```
{
 "hostelName": "SIMATS Hostel",
 "rooms": [
  {
   "roomNumber": 101,
   "residents": [
    {
     "name": "Arun Kumar",
     "registerNumber": "192311001",
     "phoneNumber": "9876543210"
    },
    {
     "name": "Divya",
     "registerNumber": "192311002",
     "phoneNumber": "9876543211"
    }
   ]
  }
 ]
}
```


Advantages:

- Supports nested data structures.
- Uses readable field names.

- New optional fields can be added more easily.
- Suitable for communication between different programming languages.

Disadvantages:

- Usually requires more storage than CSV.
- Requires parsing and conversion.
- Often needs an external Java library such as Jackson or Gson.

6.2 Comparison of Serialization Approaches

Feature	Java Serialization	CSV	JSON
Storage format	Binary	Text	Text
Readability	Low	High	High
Object relationships	Supports object graphs	Requires manual representation	Supports nested structures
File size	Often compact, but includes Java metadata	Usually compact for tabular data	Often larger due to field names and syntax
Java dependency	Strongly tied to Java classes	Language-independent	Language-independent
New fields	Depends on serialVersionUID and compatibility	Requires header/schema handling	Optional fields can be added more easily
Password protection	Requires fields to be excluded or custom handling	Sensitive fields must be omitted	Sensitive fields must be omitted
Best suited for	Java-specific object persistence	Tabular records and exports	Structured data exchange

6.3 Time Complexity Analysis

Let n represent the number of resident objects.

Java Serialization

Serialization traverses the Hostel object graph and writes its eligible objects.

Time complexity:

O(n)

Deserialization also reconstructs the object graph.

Time complexity:

O(n)

This assumes the number of rooms and their relationships grow proportionally with the number of residents.

CSV

The application visits each resident, writes the fields, and separates them using delimiters.

Saving complexity:

O(n)

Loading complexity:

O(n)

JSON

The application converts each resident and its fields into JSON.

Saving complexity:

O(n)

Loading complexity:

O(n)

The actual processing time also depends on the number of characters, parsing library, and object relationships.

6.4 Space Complexity Analysis

Java Serialization

The serialized object graph is stored in a binary file.

The additional stream-buffer memory is generally:

O(B)

where B is the serialization buffer size.

The object graph itself already occupies memory while it exists in the application.

CSV

If records are written one at a time using a buffered writer, additional working memory can remain approximately:

O(B)

If the complete file is loaded into memory, space increases with the total file size.

JSON

A streaming JSON writer can also use approximately:

O(B)

additional working memory.

However, constructing the entire JSON document as one in-memory String requires memory proportional to its total size.

6.5 How Does Each Approach Handle a New Field?

Java Serialization:

A new field can be added while retaining a compatible serialVersionUID. Old files receive the default value for that field.

CSV:

A new column can be added to the header. Old rows may not contain the new column, so the application must provide a default value during reading.

JSON:

A new property can be added to the JSON object. Older records that do not contain that property can be assigned a default value by the application or parsing configuration.

6.6 Engineering Decision

For the hostel management application, JSON is a suitable alternative when the data contains nested relationships such as Hostel, Room, and Resident.

Java serialization is convenient when the application stores and restores Java object graphs within a controlled Java environment.

CSV is more appropriate for tabular information, reporting, and spreadsheet exports.

Passwords and live database connections should not be stored in any of these formats.

7. Final Result

The fields required for hostel persistence were identified.

The Hostel, Room, and Resident object graph was designed for serialization.

The password and database Connection fields were marked transient because sensitive credentials and live database resources should not be restored directly from a serialized file.

A Java program was implemented using ObjectOutputStream and ObjectInputStream.

The effect of adding a phone number field was explained using serialVersionUID and default field values.

Finally, Java serialization was compared with CSV and JSON using file size, save/load complexity, memory requirements, and compatibility with new fields.

Thus, the task demonstrates object-graph serialization, transient fields, and version management in Java.